

Affine-Transformed Ray Alignment for Fast Ray Traversal

Yuki Nishidate
Keio University
Japan
yuki_nishidate@keio.jp

Issei Fujishiro
Keio University
Japan
fuji@ics.keio.ac.jp

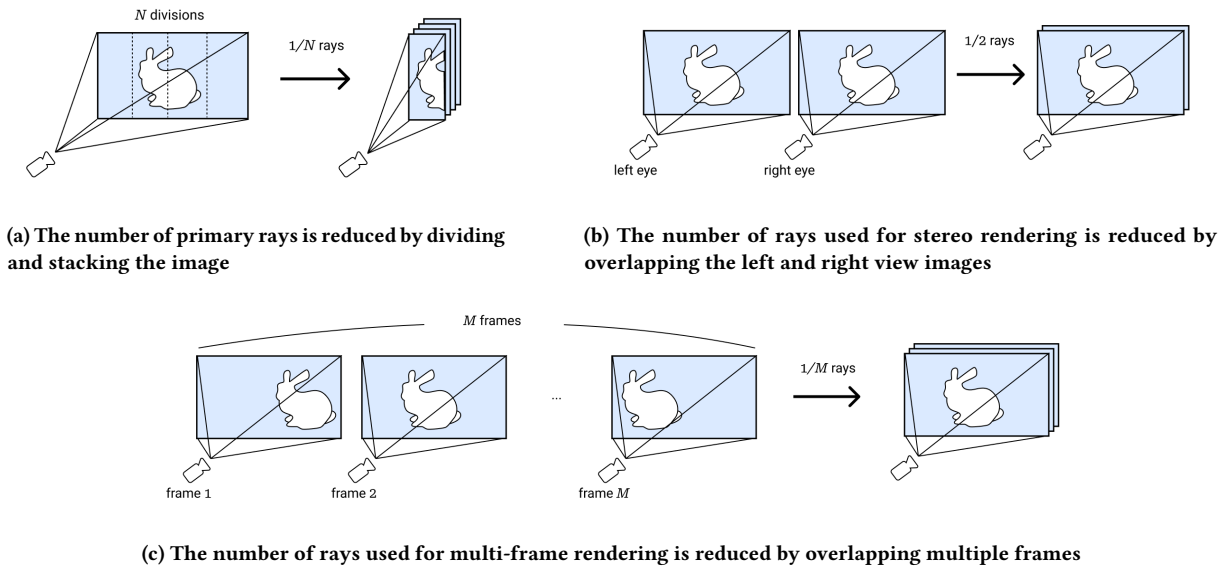


Figure 1: Application of affine-transformed ray alignment to three rendering problems.

ABSTRACT

In recent years, ray tracing has begun to be used for real-time rendering. However, the number of rays that can be emitted in real time is limited due to the high time complexity of ray tracing. In this paper, we propose affine-transformed ray alignment as a novel method of reducing time complexity. Using affine transformations, the proposed method aligns multiple rays to form a single ray while maintaining the intersection points obtained from the scene, thereby speeding up ray tracing without introducing errors. We also propose application of our method to the problems of primary ray traversal, stereo rendering, and multi-frame rendering and empirically prove that time complexity can be reduced by these simple implementations.

CCS CONCEPTS

• Computing methodologies → Ray tracing.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA '22 Technical Communications, December 6–9, 2022, Daegu, Republic of Korea
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9465-9/22/12...\$15.00
<https://doi.org/10.1145/3550340.3564228>

KEYWORDS

rendering, ray tracing, bounding volume hierarchy

ACM Reference Format:

Yuki Nishidate and Issei Fujishiro. 2022. Affine-Transformed Ray Alignment for Fast Ray Traversal. In *SIGGRAPH Asia 2022 Technical Communications (SA '22 Technical Communications)*, December 6–9, 2022, Daegu, Republic of Korea. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3550340.3564228>

1 INTRODUCTION

Ray tracing is an algorithm used to render photorealistic images. In recent years, ray tracing has been used for real-time rendering in computer games, architectural visualization, and other applications. However, ray tracing is computationally expensive, and the number of rays that can be emitted in real time is limited.

Many methods have been proposed to reduce the number of rays and significantly speed up the process, but these methods have the inherent drawback that the intersections needed are not produced, which causes errors in the rendering result.

In this paper, we propose affine-transformed ray alignment as a novel method of reducing the time complexity of ray tracing. The method reduces the number of rays by aligning them to form a single ray using affine transformations. Subsequently, polygons are instantiated using the same affine transformation to obtain multiple

intersections with a single ray. Thus, ray traversal can be accelerated without introducing errors.

The acceleration principle of our method provides a new perspective on the tradeoff between the number of rays and the number of polygons. The versatility and effectiveness of the method can be demonstrated by applying it to multiple usage scenarios, such as primary ray traversal, stereo rendering, and multi-frame rendering (Figure 1).

2 RELATED WORK

The time complexity of ray tracing depends largely on the number of intersection tests between the scene and the rays. Here, we review existing methods in terms of scene data structure and ray reduction.

Bounding volume hierarchy (BVH) [Clark 1976] is a tree structure used for accelerating ray tracing. When testing ray intersections, the BVH is recursively traversed from the root, which reduces the order of the time complexity to the logarithm of the number of polygons. Our method takes advantage of the BVH property to achieve greater speed.

A typical approach to reducing the number of rays is to introduce approximation. For example, in path tracing, speedup can be achieved by reducing the number of ray samples [Rushmeier and Ward 1994]. The problem with this is that the number of intersections is reduced at the cost of increased bias or variance. The key advantage of our method is that the number of intersections obtained does not change when the number of rays is reduced.

Our method is similar to the method of sorting rays into coherent packets [Garanzha and Loop 2010]. The difference is that our method actually reduces the number of rays.

3 AFFINE-TRANSFORMED RAY ALIGNMENT

Affine-transformed ray alignment is a method of reducing the number of rays by aligning multiple rays to form a single one using affine transformations. Normally, intersections are lost when the number of rays is reduced, but with this method, multiple intersections can be evaluated using only a single ray. Our method is extremely easy to implement, requiring no new data structures or new sampling algorithms. All that is required is proper instancing of the scene.

The procedure is divided into four steps. First, an affine transformation matrix is computed, which is required to align one ray to another. Note that, it is not necessary to multiply the ray origin and direction by the affine transformation matrix. Next, the polygons in the scene are instanced and transformed using the computed transformation matrix. The corresponding ray index is set to each of the instances. The BVH is then constructed so as to include the instanced polygons. Finally, the aligned ray is emitted, and multiple intersections are evaluated while the BVH is traversed.

Figure 2 shows two rays aligning to form one using an affine transformation, and one polygon is duplicated to become two using the same matrix. This maintains the relative positions of the rays and polygons before and after transformation and makes the two resulting intersections coincide.

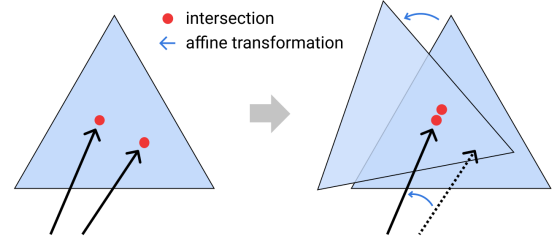


Figure 2: The concept of affine-transformed ray alignment. The same affine transformation is applied to both rays and objects to obtain the same intersections as before the transformation while reducing the number of rays.

Principle of Acceleration. When a BVH is used, the number of intersection tests in ray tracing is ideally expressed by

$$\log(N_p) \times N_r,$$

where N_p and N_r denote the numbers of polygons and rays, respectively. This expression shows that N_r affects the number of intersection tests linearly, whereas N_p affects them logarithmically. Affine-transformed ray alignment reduces the number of rays and increases the number of polygons. For example, if the rays are halved and the polygons are doubled, the number of intersection tests changes as follows:

$$\log(N_p \times 2) \times \frac{N_r}{2},$$

implying that ray reduction reduces the number of intersection tests linearly, while duplicating polygons increases the number of intersection tests logarithmically. This leads to reduced intersection tests and accelerated rendering.

Applicable Conditions. Before rendering, affine-transformed ray alignment requires the determination of affine transformations between multiple rays. In other words, the method is applicable only to rays whose origins and directions are determined prior to rendering. In order for this method to be most effective, the situation should be such that many pairs of rays can be aligned using a single affine transformation.

4 APPLICATIONS

In this section, we provide examples of the application of affine-transformed ray alignment to various rendering problems. We describe its applications to primary ray traversal (Section 4.1), stereo rendering (Section 4.2), and multi-frame rendering for VR applications (Section 4.3).

4.1 Primary Ray Traversal

The primary ray is the first ray emitted from the viewpoint. In this case, since the origin and directions of all the rays are determined prior to rendering, the applicability conditions are satisfied.

Consider the following example: If the screen is divided into two parts and the rays on the right side of the screen are transformed to the left in a projected coordinate system, they overlap the rays on the left side of the screen, thus halving the number of rays, as illustrated in Figure 3. Since the rays need only be emitted to the

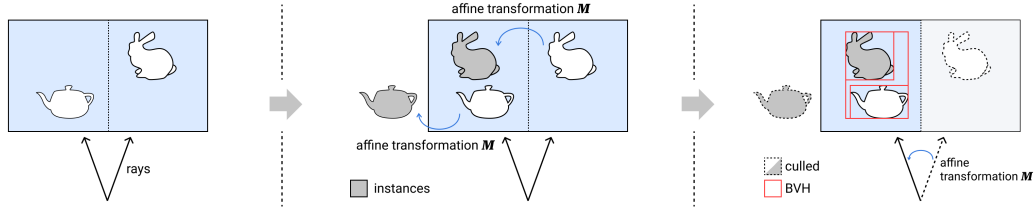


Figure 3: An example of affine-transformed ray alignment used to reduce the number of primary rays. Before rendering, instances of the objects are generated and shifted to the left by half the distance of the image before the BVH is constructed. When rendering, rays are emitted only on the left half of the screen, and two intersections are obtained from a single ray, which provides the same results as those produced using the original rays emitted to the original image.

left half of the screen, objects not included in the frustum can be culled before the BVH is constructed. This further reduces the time complexity of the intersection tests and BVH construction.

The screen can be divided horizontally or vertically, and the number of divisions can be changed. However, for high spatial coherence, this paper assumes that the screen is divided only horizontally (Figure 1 (a)).

When the screen is divided horizontally into two parts, the affine transformation matrix M for aligning the right rays to the left rays is expressed by

$$M = V^{-1} \begin{bmatrix} 1 & 0 & -\frac{1}{2}w & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} V,$$

where V denotes a view matrix (rotation and translation only), w denotes the screen width, and the distance from the camera to the screen is assumed to be 1. This matrix needs to be computed only once, so the computational overhead is negligible.

4.2 Stereo Rendering

In stereo rendering, a ray emitted from the right eye can be aligned to that emitted from the left eye by translating the interpupillary distance. This halves the number of rays and speeds up rendering, as shown in Figure 1 (b).

The affine transformation matrix M required to align the rays is

$$M = V^{-1} \begin{bmatrix} 1 & 0 & 0 & -d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} V,$$

where d is the interpupillary distance and V is the view matrix.

4.3 Multi-Frame Rendering

Our method can also be applied to multi-frame rendering by temporally overlapping scenes in adjacent frames and constructing a single BVH (Figure 1 (c)).

In a scene where the object and camera are dynamic, the affine transformation matrix M required to align the rays of the next frame to the rays of the current frame is expressed by $M = C^{-1}$, where C is the transformation matrix of the camera between the current

frame and the next frame. Note that the affine transformation matrix multiplied over the objects also requires the model transformation matrix for the next frame of each object.

5 RESULTS

In this section, we empirically evaluate the proposed method by considering its effectiveness in three specific applications. All experiments were conducted on a computer with an Intel Core i7-8700K 3.7 GHz CPU with 48 GB of RAM and a Windows 10 x64 operating system. The codes were implemented on the CPU using Embree [Wald et al. 2014], one of the most efficient ray-tracing kernel libraries, and the rendered images contain $1,024 \times 1,024$ pixels. In addition, GPU implementations were also performed in the experiment evaluating the use of the method in stereo rendering. A comparison of CPU and GPU implementations is provided in the supplementary material. Four sample scenes [McGuire 2017] were used: a Mitsuba with 61,600 polygons, a Fireplace room with 143,173 polygons, a Sponza with 262,267 polygons, and a San Miguel with 5,617,451 polygons.

5.1 Primary Ray Traversal

Figure 4 (a) shows the rendering time results measured for different numbers of horizontal divisions with frustum culling. The proposed method resulted in faster rendering of all scenes. Basically, it was found that the higher the number of divisions, the greater the acceleration. However, at 1024 divisions, the quality of the BVH deteriorated in the San Miguel due to too many overlapping polygons, which slowed down the speed. The maximum accelerations were 2.42 times with the Mitsuba, 1.40 times with the Fireplace room, 1.54 times with the Sponza, and 1.80 times with the San Miguel.

Figure 4 (b) shows the results of measuring the construction time. With the proposed method, the construction time increased as the number of divisions increased, but if the number of divisions was not extremely large, culling caused the effect to remain small. The details of memory usage are available in the supplementary material. The sum of rendering time and build time was the shortest at 128 to 256 divisions, based on the assumption that the BVH is built at every frame in a dynamic scene.

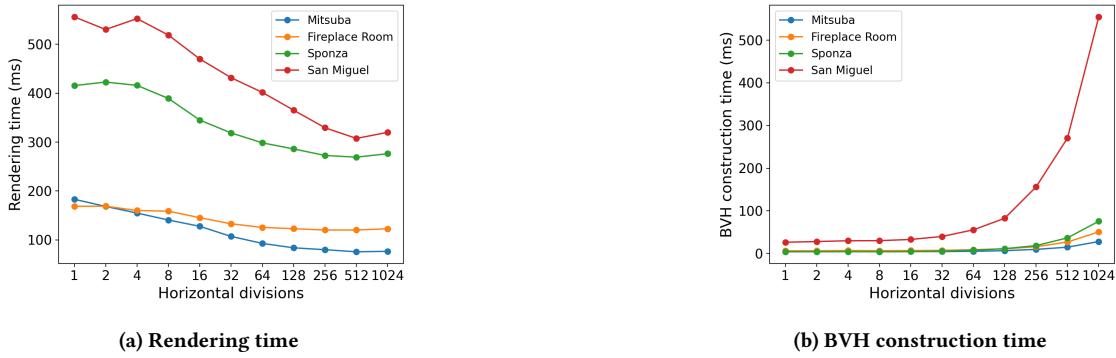


Figure 4: Results of primary ray traversal.

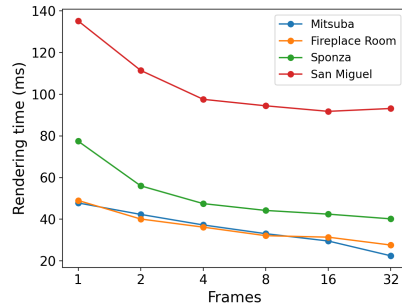


Figure 5: Rendering time in multi-frame rendering.

Table 1: Rendering time in stereo rendering.

	Mitsuba	Fireplace	Sponza	San Miguel
Naive	100.8 ms	98.00 ms	149.4 ms	271.5 ms
Ours	65.50 ms	65.83 ms	99.42 ms	182.2 ms

5.2 Stereo Rendering

To test the effectiveness of the proposed method with interactivity, the experimental conditions were set up so that scene objects could be moved and deformed dynamically and the viewpoint position could be moved dynamically.

Table 1 compares the rendering time results using the proposed method and a naive method that emits rays to both the left and right views. The increase in speed varied from 1.49 times to 1.54 times depending on the scene.

5.3 Multi-Frame Rendering

In this experiment, we assumed that the entire scene moved in every frame and all polygons were duplicated.

The rendering time results measured for the naive method and the proposed method are presented in Figure 5. In the proposed method, the time required was divided by the number of simultaneously rendered frames; in other words, the rendering time is the time per frame. In general, the more frames, the faster the speed. In complex scenes, however, the speed slowed down at 32 frames.

6 LIMITATIONS

In practical ray tracing where the scene contains only opaque objects, the search can be terminated as soon as the nearest hit point is found. Unfortunately, it is not possible to choose this optimization using our method. This problem can be addressed by terminating the traversal when the same number of closest hit points as aligned rays is obtained.

Owing to the applicable conditions, our method is difficult to apply to secondary or subsequent rays. Our method could be used to overcome this challenge by correlating the ray reflection directions, but the issue of statistical bias requires addressing.

7 CONCLUSION

This work provides a new perspective on the fundamental issue of ray tracing acceleration by proposing a method of reducing the number of rays through affine transformation. The proposed method is a step toward unifying the hybrid rendering of rasterization and ray tracing. Furthermore, we empirically proved the applicability of the proposed method to primary ray traversal, stereo rendering, and multi-frame rendering, to produce faster rendering with less increase in spatial complexity and less impact on BVH construction time.

ACKNOWLEDGMENTS

This work has been partially supported by Grant-in-Aid for Challenging Research (Pioneering) JP20K20481 and the Research Grant of Keio Leading-edge Laboratory of Science & Technology.

REFERENCES

- James H. Clark. 1976. Hierarchical Geometric Models for Visible Surface Algorithms. *Commun. ACM* 19, 10 (1976), 547–554. <https://doi.org/10.1145/360349.360354>
- Kirill Garanzha and Charles Loop. 2010. Fast Ray Sorting and Breadth-First Packet Traversal for GPU Ray Tracing. *Computer Graphics Forum* 29, 2 (2010), 289–298. <https://doi.org/10.1111/j.1467-8659.2009.01598.x>
- Morgan McGuire. 2017. *Computer Graphics Archive*. <https://casual-effects.com/data>
- Holly E. Rushmeier and Gregory J. Ward. 1994. Energy Preserving Non-Linear Filters. In *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '94)*. 131–138. <https://doi.org/10.1145/192161.192189>
- Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, and Manfred Ernst. 2014. Embree: A Kernel Framework for Efficient CPU Ray Tracing. *ACM Transactions on Graphics* 33, 4 (2014), 143:1–143:8. <https://doi.org/10.1145/2601097.2601199>